

Voice Appointment Agent

A Brief: the design, the evidence, and the gaps

Portfolio explainer, built from the repository source

1 The project in one page

A small business loses money every time nobody answers the phone. This project is a working answer: a real telephone number a person can speak to in ordinary sentences, which books, reschedules, and cancels appointments in a real calendar.

Three off-the-shelf pieces, one piece of original work:

- **Twilio** rents the phone line, answers the call, converts speech to text, and speaks replies back.
- **Gemini 2.5 Flash** reads each caller utterance and returns structured data: what they want, and the details extracted so far.
- **SQLite** stores appointments and refuses to double-book.
- **The original work** is the state machine in between, plus the architecture that makes an LLM-driven phone system testable without spending money on every run.

The claim to make, and its exact boundary

The booking logic is thoroughly and genuinely verified: 47 free automated tests and 25 scripted conversation scenarios, all passing, all reproduced independently while writing this document.

What is *not* verified is that this survives a real human voice. The language model has only ever been shown clean, typed English; Twilio's speech-to-text output is noisier. That gap is real, and the project's README states it plainly rather than glossing it.

2 Architecture

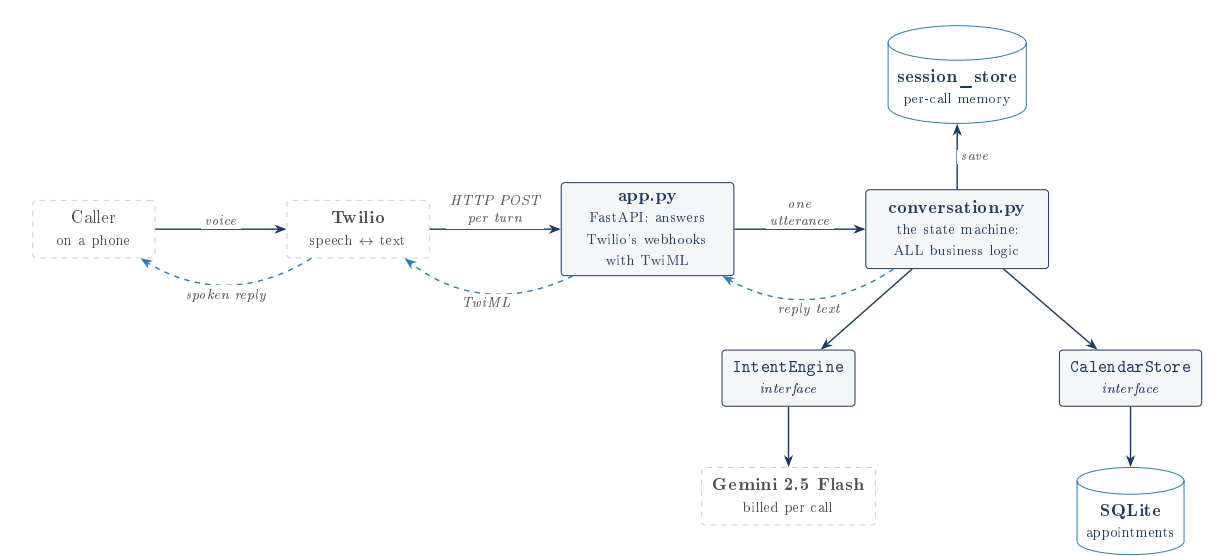


Figure 1: `conversation.py` holds every business rule and depends on *neither* Twilio *nor* Gemini. It talks only to the two interfaces. That is the whole design.

2.1 The decision everything else rests on

The conversation manager is handed its collaborators rather than creating them, and it only ever knows them as `IntentEngine` and `CalendarStore`. So the identical state machine runs in three worlds:

Where	Language model	Calendar	Cost per run
Production (uvicorn)	real Gemini, billed	SQLite file	real money
47 pytest tests	scripted double	temp SQLite	nothing
25 eval scenarios	scripted double	temp SQLite	nothing
6 marked tests	real Gemini	not involved	a fraction of a cent

Table 1: One state machine, four wirings. The expensive dependency is present exactly where it must be proven, and absent everywhere else.

Why this is the strongest thing in the project

25 scenarios, several of them five turns long, would be over 100 billed model calls per run, would take real time, and would be flaky, because language models are not deterministic. Defining what the manager *needs* rather than what it will *use* converts an unaffordable test suite into one that runs in under three seconds for free, and still exercises the real state machine, the real database, and real conflict detection.
The discipline is what makes it credible: the fake engine lives in `eval/`, outside the production package, and its own docstring says “This is a TEST DOUBLE, not a production fallback”. The shipped path always uses the real model, so the “LLM-driven” claim holds.

3 How a call works

A phone call is not one conversation from the server’s point of view; it is a series of separate HTTP requests with no memory of each other. Everything the agent knows must therefore be written to SQLite and read back every turn, keyed by Twilio’s `CallSid`. That single fact explains why `session_store.py` exists at all.

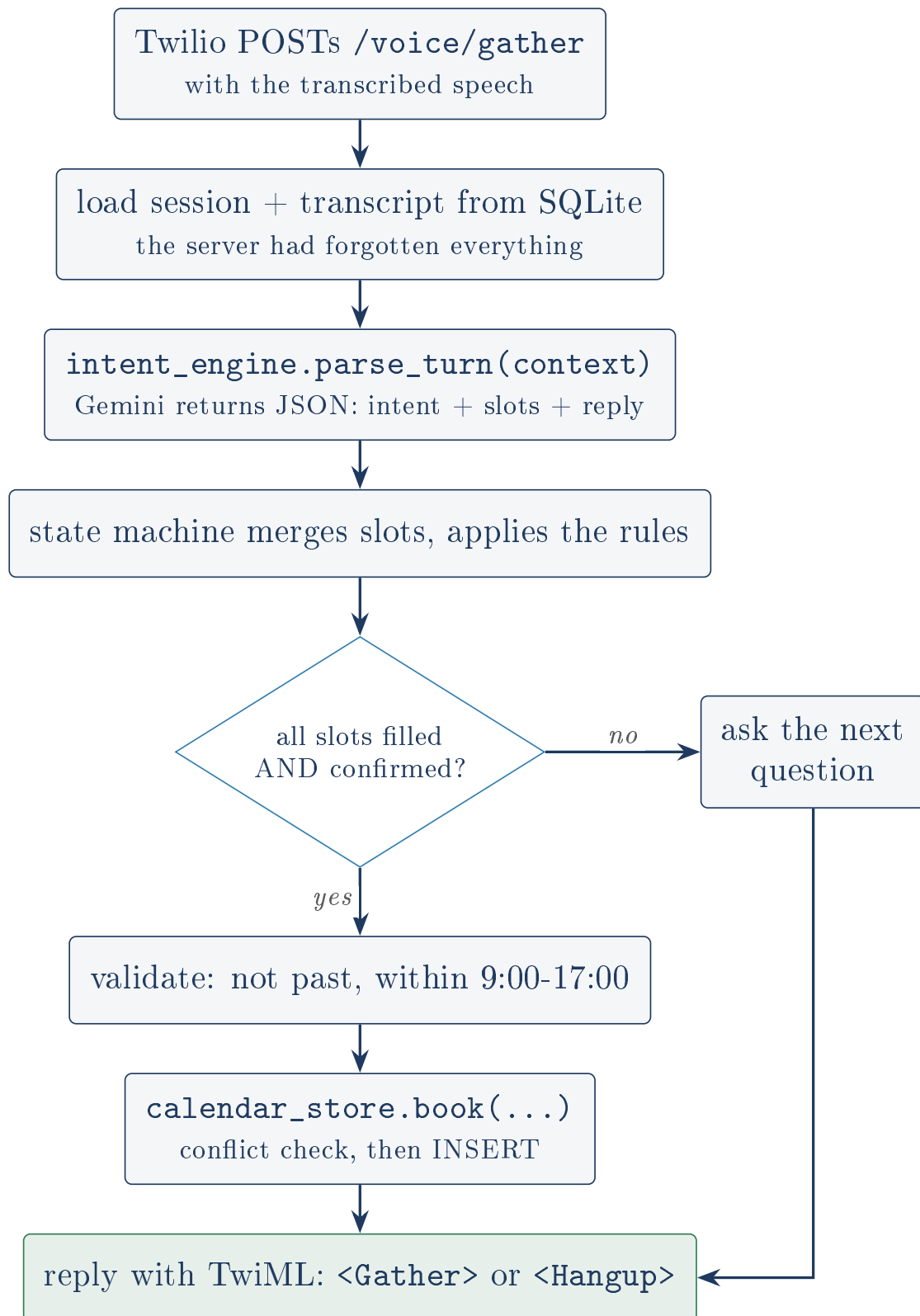


Figure 2: One turn, end to end. Repeat until booked, hung up, or handed off.

4 The engineering decisions worth defending

4.1 Turning the language model’s flagship feature off

Gemini 2.5 Flash “thinks” before answering by default. That helps hard reasoning and costs seconds. On a phone call, several seconds of dead air is a product failure: the caller assumes the line dropped. So the production configuration sets `thinking_budget=0`, plus a strict response schema and `temperature=0.1`.

The general lesson

The right configuration depends on the product, not on the model’s defaults. For a live phone turn, latency beats depth. Knowing which clever feature to switch off is engineering; leaving defaults on is not.

Equally: asking a model in prose to “please reply in JSON” sometimes yields “Sure! Here’s the JSON:” and then fails to parse. Supplying a formal `response_schema` to the API constrains the output structurally. That is the robust way to get machine-readable answers.

4.2 Reading the vendor documentation properly

Twilio’s `<Gather>` requests your server *only when it captures speech*. On silence it quietly moves on to the next instruction instead. The consequence would have been severe: every “sorry, could you repeat that?” retry path would be unreachable for a silent caller, who is precisely the caller who needs it. One attribute, `action_on_empty_result=True`, routes silence back through the same state machine. This is the class of detail that separates working telephony from a demo.

4.3 Two safety nets, for two different failures

- **Retry cap (3 consecutive unclear turns)** catches the bad line and the noisy room.
- **Turn cap (14 total turns)** catches what the retry cap cannot: a caller whose every turn is perfectly intelligible but never completes anything. Say “sometime in the morning” forever and the retry counter resets every turn, because each turn was a valid continuation. Without the second cap the call never ends.

The test for the turn cap asserts `retry_count == 0`, proving *which* mechanism saved the call rather than merely that something did. That is careful test design.

4.4 Making time an input

The conversation manager accepts an injectable clock (`now_fn`) rather than calling `datetime.now()`. Without this, a test asserting “the appointment lands on 2026-07-15” passes today and silently starts failing once that date passes, for reasons unrelated to any code change. Every scenario pins time to `FIXED_NOW = 2026-07-06 13:00`, chosen at mid-afternoon so a same-day “past” time can be tested without also tripping the business-hours rule.

4.5 Half-open intervals

Conflict detection is `a_start < b_end` and `b_start < a_end`. Strictly less-than, so an appointment ending at 9:30 and one starting at 9:30 are back to back rather than in conflict, which is what a busy clinic needs. Using `<=` would refuse a perfectly good booking. There is a dedicated scenario for the boundary case.

5 The evidence

Every free number below was independently reproduced while writing this document, by installing the pinned requirements into a clean virtual environment and running the suite. The 7 tests that

spend money were collected but **deliberately never executed**.

Claim	Observed	Verdict
54 tests collected	54 tests collected	confirmed
47 free, 7 billed	47 passed, 7 deselected	confirmed
25/25 scenarios pass	25/25 scenarios passed	confirmed
Clean venv install	exit code 0	confirmed
Latency 1.8 s with thinking off	no artefact in repo	<i>owner's record</i>
Live public-tunnel booking	no artefact in repo	<i>owner's record</i>

5.1 What the 25 scenarios actually test

Precisely, because the honest answer is narrower than it sounds

Genuinely exercised: the real state machine, a real SQLite calendar on a real temporary file, real conflict detection, real business-hours and past-date validation, real retry and turn caps. Coverage spans booking (one-shot and multi-turn), rescheduling, cancelling, conflicts, boundary-adjacent slots, ambiguous times and dates, unclear speech, both safety caps, abandoned calls, and mid-call changes of mind.

Substituted: the language model, replaced by the regex-based scripted double.

Not involved at all: Twilio, FastAPI, HTTP, TwiML. Utterances are fed straight into the manager. Those layers are covered separately by 7 endpoint tests.

So “25/25” is a strong statement about the booking *logic* and no statement about speech recognition. That framing is the one to offer first.

5.2 The tests were themselves tested

A passing test proves nothing unless it can fail. The README claims the safety caps were deliberately broken to confirm the scenarios noticed. That was re-verified here in a scratch copy, leaving the repository untouched:

Sabotage	Result	Caught by
retry cap 3 → 999	24/25	unclear_speech_retry_cap_handoff
turn cap 14 → 9999	24/25	max_turn_cap_handoff
< → <= in overlap	24/25	boundary_adjacent_appointments_no_conflict

Table 2: Each sabotage killed exactly the one scenario named for it, and nothing else. The suite is load-bearing, not decorative.

The scripted engine cannot fail to understand

The fake engine was written to handle exactly the phrases the scenarios use; its docstring concedes this. So no scenario can ever surface a misunderstanding. The 25 scenarios test the state machine *given perfect understanding*. Real understanding is tested only by the 6 marked Gemini tests, against clean typed English, and never against messy speech output. This is the correct trade, and the project is open about it. Say it first: “25/25 proves the booking logic; I made the understanding layer a separate, smaller, honest experiment.”

6 Honest findings

Found by reading the source and confirmed by running the real code in a scratch copy. None of these appear in the README.

The webhook signature check fails open by default

The most significant issue. Twilio signs each request so you can prove it came from Twilio. The check is implemented correctly, but it is wrapped in `if base_url and auth_token:`, and `PUBLIC_WEBHOOK_BASE_URL` is blank by default, with `.env.example` instructing “leave blank for local development”.

Verified by driving the real app: with enforcement nominally *on* and no base URL, a request carrying *no signature at all* returns **HTTP 200**. Set the base URL and the same request correctly returns **403**.

There is a real reason (Twilio signs the public URL, so without it there is nothing to check against). The problem is the direction of failure: a missing setting yields silently no security rather than a refusal to start. Deploy behind a real URL and forget the variable, and the booking endpoints are open to anyone. The fix is small: refuse to start, or make unsigned requests an explicit opt-in.

Phone numbers are stored in two incompatible formats, and a test locks it in

Twilio gives caller ID as `+15551112222`; speech is parsed to bare digits `5551112222`; lookup is an exact string match. Verified: an appointment booked after the caller *spoke* their number cannot be found when that person phones back. The caller who was most helpful is the one who cannot cancel.

Sharpest detail: scenario 4 *asserts this as correct*, checking that the caller-ID lookup returns zero results. Its intent (a spoken number should override caller ID) is right; what is missing is normalising both to one canonical format on input. A good example of a test encoding an assumption rather than a requirement.

The wrong appointment is targeted when a caller has two

`list_by_phone` sorts furthest-future first, and the code takes the first match. Verified: with appointments on 2026-08-01 and 2026-09-01 under one number, “cancel my appointment” targets **September**, keeping August. The agent does read the date back, so an attentive caller can object, but “my appointment” in ordinary speech means the next one. No scenario covers a caller with two active appointments, which is exactly why the suite stays green.

The Google Calendar adapter has never run, and reading it shows why that matters

Fully implemented against the real Calendar API, selected automatically if credentials are set, but no credentials were ever provided. The README says so. Reading it reveals a concrete defect: `cancel()` deletes the event and then fabricates a return value using `datetime.now()`, and the conversation manager speaks that value back to the caller. On this backend the agent would announce *the current time* instead of the appointment’s real time. Invisible on SQLite, where `cancel()` returns the true row. That is the predictable cost of shipping code you cannot run.

Smaller items

- `find_conflict` loads *every* booked appointment ever made into Python on each booking, with no time filter and no index. Invisible at this scale; a one-line `WHERE` clause away from correct.
- `book()` checks for conflicts and inserts as two statements with no transaction, so two simultaneous bookings could both pass the check. One phone line makes it vanishingly unlikely, but “why is your conflict check not atomic?” is a fair question.

- `SessionState.stage` is declared, serialised into every row, and never read or written anywhere. Dead field from an earlier design.
- `cancel()` on an already-cancelled appointment silently succeeds, though the interface documents it as raising. Its sibling `reschedule()` does check. Inconsistent.
- No timezones anywhere; every time is one implicit local zone. Acknowledged in the README.

Three claims rest on the owner's word

The 1.8 second latency measurement, the 56 thinking-token figure, and the live public-tunnel booking run all appear only in README prose; no log or artefact survives in the repository. The tunnel run is the strongest verification claimed and was cleaned up afterwards by design. Nothing suggests these are untrue, and the README's precision elsewhere argues the other way; it even labels its own cost figure "an estimate from per-token rates, not a verified line-item invoice", which is exactly right. But "I ran it and here is the log" and "I ran it and cleaned up" are different, and an interviewer may probe. Next time, commit the log.

7 Security posture

Secret hygiene is genuinely good. `.env` is a symbolic link pointing outside the repository to a private secrets directory, so no credential is ever in the working tree. The full git history (all 46 objects) was scanned for Twilio SIDs, Google API keys, private-key blocks, and hard-coded tokens: clean. `.env.example` carries placeholders only. Every SQL query is parameterised, which matters because a caller's spoken name reaches the database. Configuration values are never logged.

The one real weakness is the fail-open signature check described above.

8 Summary judgement

Lead with these

1. **The interface seam.** Two small abstract classes turn an expensive, non-deterministic system into one with 72 free, deterministic checks. Best thing here.
2. **Turning thinking mode off**, having measured it. Product engineering, not cargo-culted defaults.
3. **The tests were tested.** Mutation testing reproduced exactly.
4. `actionOnEmptyResult`. Evidence of reading the documentation properly.
5. **The README's candour** about its own gaps exceeds most production documentation.

Raise these before someone else does

1. The signature check fails open without the base URL.
2. Two phone formats, with a test that locks the bug in.
3. The furthest-future appointment is targeted, not the soonest.
4. "25/25" means the logic is right *given perfect understanding*.
5. No timezones; the Google adapter has never run; no real voice call has been made.

The honest one-line verdict: a well-architected demonstration of a real pattern, tested far more rigorously than most portfolio projects, whose remaining gap (a real human voice on a real line) is the one the author names first himself.

Built from the repository source. Every claim is quoted from the code, verified by running it in an isolated scratch copy, or explicitly labelled as the owner's own record. No phone call was placed and no billed API test was executed in the making of this document.